

软件质量保证与测试

——编码过程中的测试



编程与测试的联系

- 代码规范性、一致性和可测性

主要的工作包括：

- 参与代码的检查、走查等。
- 参与单元测试、集成测试。
- 进行 Ad-hoc 测试，熟悉产品，完善测试用例。

编程与测试的联系

代码的审查

静态模式，代码中的缺陷60%以上可以通过代码审查(包括互查、走查、会议评审等形式)发现出来；代码审查必须依靠具有软件开发和编程经验的技术人员集体进行。

编程与测试的联系

代码的审查标准

- (1) **正确性**: 代码逻辑必须正确, 能够实现预期的功能。
- (2) **清晰性**: 代码必须简明、易懂, 注释准确没有歧义
- (3) **规范性**: 代码必须符合企业或部门所定义的共同规范, 包括命名规则、代码风格等。
- (4) **一致性**: 代码必须在命名上(如同功能的变量尽量采用相同的标示符)、风格上都保持统一。
- (5) **高效性**: 代码不但要满足以上性质, 而且需要尽可能地降低代码的执行时间。

编程与测试的联系

代码的审查范围

- (1) **业务逻辑的审查**: 主要审查是否是按照实现设计的规格说明书展开的编程, 逻辑是否正确且简单, 思路是否清晰?
- (2) **算法的效率**: 无论是内存处理还是统计排序算法, 或者SQL查询语, 精心设计其算法, 确定最优化的方法。
- (3) **代码风格**: 代码的命名规则、注释行、嵌套、格式等直接影响了软件的可读性、可维护性和可靠性等方面的质量
- (4) **编程规则**: 包括语句的完整性、数据定义的准确性、常量和变量定义、参数调用、参数使用、内存管理、逻辑表达式等。

编程与测试的联系

代码的审查方法

- (1) **互查**：处在相同模块或相近模块的编程人员互相检查对方代码，方法简单有效，容易开展。
- (2) **走查**：对于新人编写的程序，一般需要采用走查的方式，从头到尾将写好的程序检查一遍。
- (3) **会议评审**：正式，主要是审查关键性的代码。

编程与测试的联系

内存分配和管理：

内存的及时释放和避免缓冲区溢出。

程序性能的审查：

- (1) 算法的优化，一些关键的排序、分类和计算
- (2) 减少创建对象
- (3) 减少循环体的执行代码
- (4) 提高处理异常出错的效率：try/catch进行异常处理使用方便，但避免过度使用异常处理
- (5) 减少IO操作时间：

单元测试例子

```
public String getUserRole (HttpServletRequest request)
{
    String userRole="";
    String  userName=request.getParameter("userName");
    if(userName.equals("admin")) {
        //这是系统初始化时默认的管理员账号, 如果是, 则做以下的验证操作...
    }
    return userRole;
}
```

- 空指针保护错误

```
public int getUserAge(HttpServletRequest request) {
    int age=0;
    String userAge =request.getParameter("userAge");
    if(userAge !=null)
    {
        age =Integer.parseInt(ugerAge);
    }
    return age;
}
```



```

public static void writeStringFile(File file, String writeContent,
    String encoding) throws FileOperatorException {
    FileOutputStream fos = null;
    try {
        if (!file.exists()) {
            file.createNewFile();
        }
        fos = new FileOutputStream(file);
        fos.write(writeContent.getBytes(encoding));
    } catch (Exception ex) {
        throw new FileOperatorException(ex);
    } finally { //如果没有finally下面的段
        if (fos != null) {
            try {
                fos.close();
            } catch (IOException ioe) {
                throw new FileOperationException(ioe);
            }
        }
    }
}

```

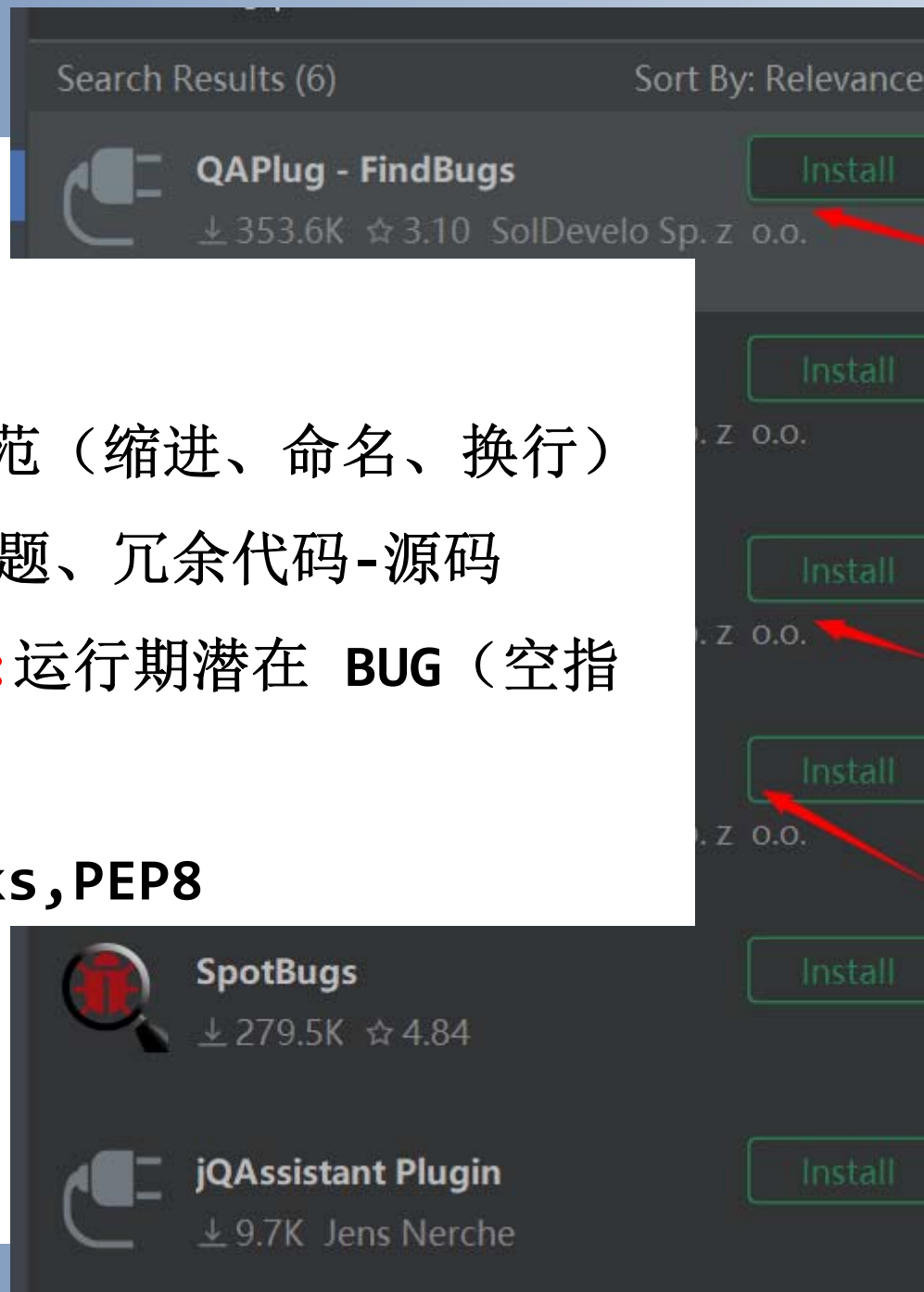
- 资源不合理利用

代码静态检测工具

支持Java语言检测工具

- (1) **CheckStyle**: 编码格式规范（缩进、命名、换行）
- (2) **PMD**: 代码坏味道、设计问题、冗余代码-源码
- (3) **SpotBugs (FindBugs)**: 运行期潜在 **BUG**（空指针、资源未关闭）-编译

支持Python: Pycharm, Pyflaks, PEP8



CheckStyle

代码风格检查工具。Checks能够根据代码规范自动地检查代码的风格，能够检查的主要内容包括以下几个方面。

(1)注解；(2)代码块；(3)类设计；(4)编码问题；(5)文件头；(6)Import 语句；(7)Javadoc 注释； 8)复杂度；(9)混合检查；(10)修饰符；(11)命名约定；(12)正则表达式；(13)体积大小；（14）空白字符

代码静态检测工具

PMD：开源的静态代码分析工具，用于检查Java、JavaScript、PLSQL和其他语言的代码中的潜在问题。

如未使用的变量、未使用的方法、无效的if语句等。

PMD通过解析代码，并应用各种规则来检查代码中的潜在问题。这些规则可以根据代码质量标准进行配置，并且可以自定义规则集合。

PMD提供了多种输出格式，包括控制台输出、HTML、XML和JSON格式，方便用户进行代码分析和结果处理。

在IDE中进行代码分析和问题修复。

PMD 7类规则集

- (1) **最佳实践** (Best Practices): 检查代码是否遵循了公认最佳实践
- (2) **代码风格** (Code Style): 强制遵守特定的代码风格
- (3) **设计** (Design): 帮助用户发现设计问题的规则集
- (4) **文档** (Documentation): 检查代码中的注释文档。
- (5) **易错倾向** (Error Prone): 用于检测损坏的、容易混淆或出现运行错误的代码
- (6) **多线程** (Multithreading): 检查代码中处理多个执行线程的问题
- (7) **性能** (Performance): 标记次优代码的规则集。

代码静态检测工具

SpotBugs (FindBugs) 检查代码缺陷

Findbugs是一个静态分析工具，它检查类或者jar文件，将字节码与一组缺陷模式进行对比以发现可能的问题。

可以帮助改进代码质量。可以生成报告

SpotBugs (FindBugs) 9种类型

- 1) Bad practice 不良的实践
- 2) Dodgy code可疑或者危险的代码
- 3) Malicious code vulnerability恶意的代码漏洞
- 4) Correctness正确性问题
- 5) Performance 性能问题
- 6) Multithreaded correctness 多线程正确性
- 7) Experimental 经验性问题
- 8) Security安全性问题
- 9) Internationalization 国际化问题

代码级的动态测试——单元测试

什么是单元测试

尽可能早的发现软件中存在的错误，降低软件质量成本。

单元
测试

功能
测试

集成
测试

集成
测试

单元测试对象是最小单位（函数、类方法、功能模块、组件）能够实现一个特定功能，具有一定独立性或者又通过明确接口定义与其他单元联系起来。

单元测试目的： 检验软件单元能否正确地实现其功能，满足性能和接口要求。（验证代码和详细设计说明一致性）

代码级的动态测试——单元测试

为什么要进行单元测试

单元测试中的单元是软件系统或产品中可以被分离又能被测试的最小单元。如果某个组件比较大，可以进一步分离某些部分，直至分离到一个可接受的程度，这取决于可测试性（类、函数或某个程序子过程）。

单元测试就是确保构成软件系统的各个成分得到足够的测试，只有通过了单元测试，集成测试和系统测试才能顺利实施。成为测试驱动开发或极限编程的思想，从测试、用例出发来进行软件开发，单元测试应伴随着编程过程中的每一个时刻

代码级的动态测试——单元测试

单元测试的现状

知道代码要编写测试程序，但却很少这样做。

后果是：

后期在功能测试、集成测试和系统测试会充满各种缺陷，导致维护成本升高，花费的时间和成本成指数非线性关系。

无论是新代码还是老代码，只要代码有变化，都进行单元测试。

缺陷预防措施： 严格编程规范、加强培训。

单元测试的任务

对单元功能、逻辑控制、数据和安全方面进行必要的测试。包括等测试。单元中所有独立执行路径、数据结构、接口、边界容错性。

1、单元独立执行路径的测试

在单元中应对每一条独立执行路径进行测试,这不仅检验单元中每条语句（代码行）能够正确执行,还检查下列问题:

- (1) 误解或用错了运算符优先级;
- (2) 混合类型运算;
- (3) 变量初始化错误、值错误 ;
- (4) 错误计算或精度不够;
- (5) 表达式符号错等。

这可采用判定覆盖、条件覆盖和基本路径覆盖等方法完成

单元测试的任务

2、单元局部数据结构的测试

检查局部数据结构是检查临时存储的数据在程序执行过程中是否正确、完整。

- (1) 不合适或不相容的类型说明;
- (2) 变量无初值:
- (3) 变量初始化或默认值有错;
- (4) 不正确的变量名(拼错或不正确地截断);
- (5) 出现上溢、下溢和地址异常。

单元测试的任务

3. 单元接口测试

在数据能正确输入(如函数参数调用)、输出(如函数返回值)的前提下,其他测试才有义,对单元接口的检验,不仅是集成测试的重点,也是单元测试的不可忽视的部分。

(1) 输入的实际参数与形式参数的个数、类型等是否匹配、一致; (2) 调用其他单元时所给实际参数与被调单元的形式参数个数、属性和量纲是否匹配; (3) 调用预定义函数时所用参数的个数、属性和次序是否正确; (4) 是否存在与当前入口点无关的参数引用; (5) 是否修改了只读型参数; (5) 对全程变量的定义各单元是否一致; (7) 是否把某些约束作为参数传递。

单元测试的任务

4. 单元边界条件的测试

采用边界值分析技术，针对边界值及其最靠近的两个值设计测试用例，发现新的错误，忽略，后期很难追踪。

5. 单元容错性测试

在软件构造中**强调防御式编程**，即要求在编写程序时能预见各种可能的出错条件，并对出错进行正确处理，如提示。对容错性问题：

(1) 输出的出错信息难以理解；(2) 记录的错误与实际遇到的错误不相符；(3) 在程序自定义的出错处理代码运行之前，系统已介入；(4) 异常处理不当；(5) 错误陈述中未能提供足够的定位出错信息

单元测试的任务

6. 内存分析

内存泄露会导致系统运行的崩溃, 尤其对于嵌入式系统这种资源比较乏、应用泛, 而且往往又处于重要部位的, 将可能导致无法预料的重大损失。

通过检查内存使用可以了解程序内存分配的真实情况, 发现对内存的不正常使用, 在问题出现前发现征兆, 系统崩溃前发现内存泄露错误;

发现内存分配错误, 并精确显示发生错误时的上下文情况及发生错误的缘由。

代码级的动态测试——单元测试

单元测试的方法

有效的单元测试要采用正确的方法，设计完整的测试用例。

方法： 白盒测试+辅助黑盒测试

白盒测试： 将被测程序看作一个打开的盒子了，对代码进行了全面的**逻辑分析(如条件和路径分析)**之后准确定位，进行有效的测试。

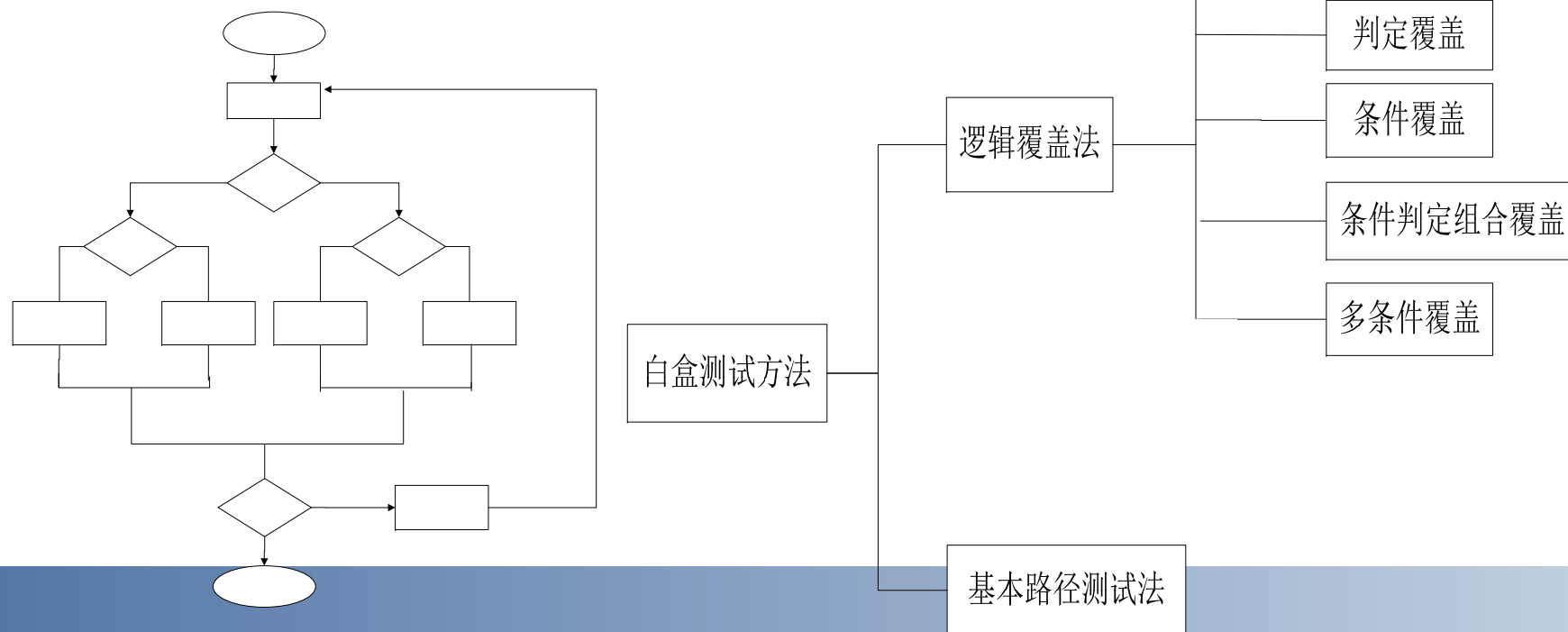
对于一个单元(一般是**几十行或几百行代码**)这种**逻辑分析**是现实的，但对于复杂的软件系统(可能是几十万行或几百万行代码)来说，根本不可能全而地完成逻辑分析，

白盒测试： **逻辑驱动法和基本路径测试法**

白盒测试

白盒测试方法根据**模块内部结构**，基于程序内部逻辑结构，针对**程序语句、路径、变量状态**等来进行测试。

单元测试主要采用**白盒测试方法**，辅以黑盒测试方法。白盒测试方法应用于**代码评审、单元程序之中**，而**黑盒测试方法**则应用于**模块、组件等大单元的功能测试之中**。



代码级的动态测试——白盒测试

白盒测试技术：

逻辑覆盖法以程序内在的**逻辑结构**为基础，根据程序的流程图设计测试用例。根据覆盖的目标不同，又可分为**语句覆盖**、**分支覆盖**、**条件覆盖**、**分支-条件覆盖**、**条件组合覆盖**和**路径覆盖**。

(1) **语句覆盖**：选择足够的测试用例，使程序中每一条**可执行语句**至少被执行一次。

(2) **判定覆盖(也叫分支覆盖)**：选择足够的测试用例，使程序中每一个分支判断的每一种可能结果都**至少被执行一次**。

(3) **条件覆盖**：选择足够的测试用例，使程序中每一个分支判断中的每一个条件的可能结果都**至少被执行一次**。

(4) **判定/条件覆盖**：选择足够的测试用例，使之同时满足判定覆盖和条件覆盖。

代码级的动态测试——白盒测试

白盒测试技术：

(5) 条件组合覆盖：选择足够的测试用例，使程序中每一个分支判断中的每个条件的每一种可能组合结果都至少被执行一次。

(6) 路径覆盖：选择足够的测试用例，使程序中所有可能的路径都至少被执行一次。

循环测试：在上下边界及可操作范围内运行所有的循环。在对单元进行白盒测试的过程中，有时需要某于被测试单元的接口，开发出相应的驱动程序/模块(Driver)和桩程序/模块(Stub)。

Driver和Stub更多的是应用在集成测试中。

白盒测试技术

【例】用不同的覆盖方法对其进行逻辑覆盖测试。

(1) Dim x, y As Integer Dim z As Double

(2) IF(x>0 AND y>0) THEN

(3) z = z/x

(4) END IF

(5) IF(x>1 OR z>1) THEN

(6) z = z + 1

(7) END IF

(8) z = y + z

白盒测试技术

对其进行逻辑覆盖测试
的第一步就是要先绘制
出它的程序流程图，如
右图所示。

路径：

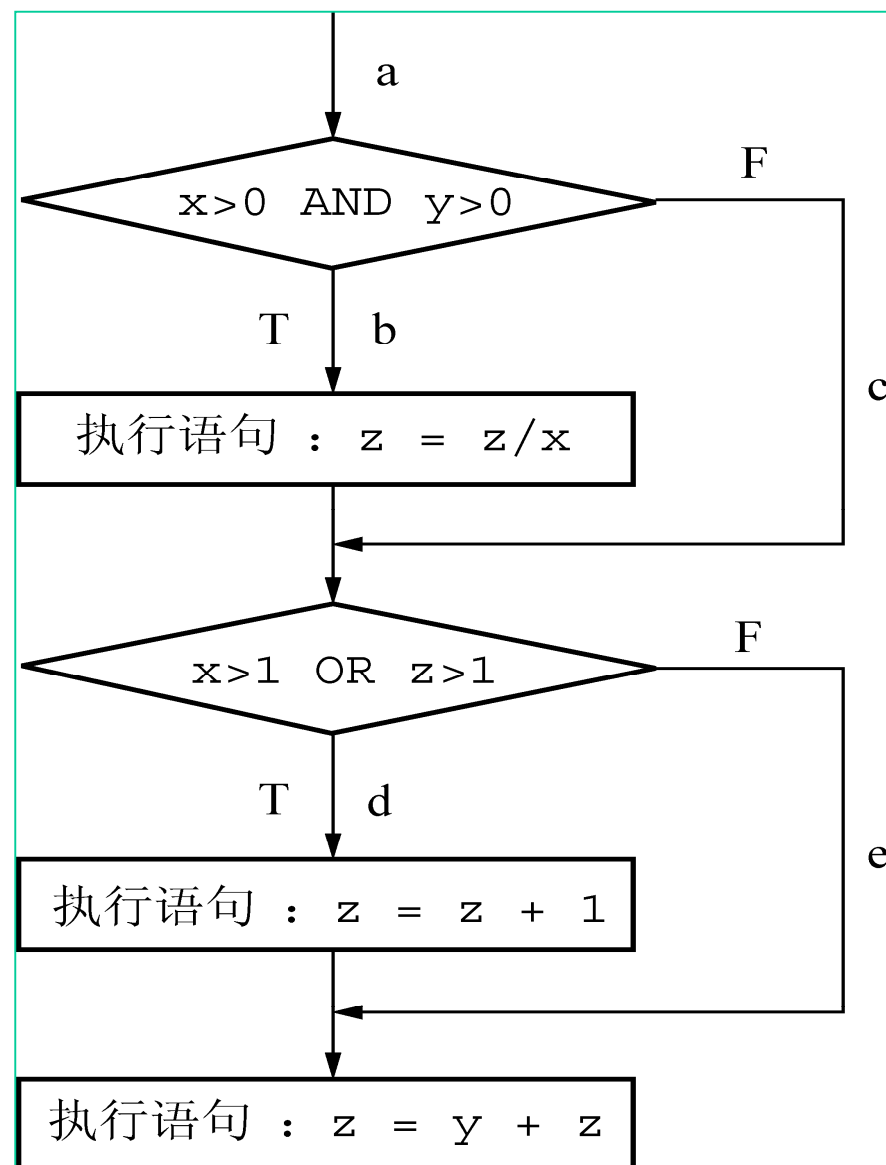
P1: abd

P2: abe

P3: acd

P4: ace

3



语句覆盖法

语句覆盖 (Statement Coverage) 又称行覆盖 (LineCoverage)、段覆盖 (SegmentCoverage)、基本块覆盖 (BasicBlockCoverage)，它是常见的覆盖方式。

语句覆盖的目的是测试程序中的代码是否被执行，它只测试代码中的执行语句，这里的执行语句不包括头文件、注释、空行等。其基本思想是设计若干个测试用例，运行被测程序，使程序中每一条可执行语句至少应该执行一次。

语句覆盖但不会考虑各种分支组合情况。

白盒测试技术

- **语句覆盖**的基本思想是，设计若干个测试用例，运行被测试的程序，使程序中的每个可执行语句至少执行一次。语句覆盖用例如下：

输入：{**x = 2, y = 3, z = 4**} 执行路径：**abd**
语句**1~8**都执行了一次。

- **分支覆盖**的思想是使每个判断的取真分支和取假分支至少执行一次。其用例为：

输入：{**x = 3, y = 4, z = 5, 分支1: T, 分支2: T**}
执行路径：**abd**

输入：{**x = -1, y = 2, z = 0 分支1: F, 分支2: F**}
执行路径：**ace**

条件覆盖：使每个判断的所有逻辑条件的每种可能取值至少执行一次。

用例设计如下：

对于判断语句 $x > 0 \text{ AND } y > 0$ ：

条件 $x > 0$ 取真为T1，取假为F1

条件 $y > 0$ 取真为T2，取假为F2

对于判断语句 $x > 1 \text{ OR } z > 1$ ：

条件 $x > 1$ 取真为T3，取假为F3

条件 $z > 1$ 取真为T4，取假为F4

条件-分支覆盖

- 条件测试用例如下表

测试用例 (输入数据)	取值条件	具体取值条件	通过路径
x=1, y=-1, z=-2	T1, F2, T3, F4	分支1: F 分支2: T	路径: acd
x=-1, y=2, z=3	F1, T2, F3, T4	分支1: F 分支2: T	路径: acd

- 分支-条件覆盖就是要同时满足分支覆盖和条件覆盖的要求。
其用例取分支覆盖的用例和条件覆盖的用例的并集即可。

测试用例 (输入数据)	取值条件	具体取值条件	通过路径
x=2, y=1, z=6	T1, T2, T3, T4	分支1: T 分支2: T	路径: abd
x=-1, y=-2, z=-3	F1, F2, F3, F4	分支1: F 分支2: F	路径: ace

条件组合覆盖

条件组合覆盖的思想是使每个判断语句的所有逻辑条件的可能取值组合至少执行一次。

编号	组合条件要求	判定条件取值	编号	组合条件要求	判定条件取值
1	T1, T2	分支1: T	5	T3, T4	分支2: T
2	T1, F2	分支1: F	6	T3, F4	分支2: T
3	F1, T2	分支1: F	7	F3, T4	分支2: T
4	F1, F2	分支1: F	8	F3, F4	分支2: F

测试用例 (输入数据)	取值条件	覆盖路径	覆盖组合
x=2, y=1, z=6	T1, T2, T3, T4	abd	1, 5
x=2, y=-1, z=-2	T1, F2, T3, F4	acd	2, 4
x=-1, y=2, z=3	F1, T2, F3, T4	acd	3, 7
x=-1, y=-2, z=-3	F1, F2, F3, F4	ace	4, 8

单选题:

如果一个判定中的复合条件表达式为 $(A > 1) \text{ or } (B \leq 3)$ ，则为了达到**100%**的条件组合覆盖率，至少需要设计多少个测试用例（ ），如果达到**100%**的条件分支覆盖，至少需要设计多少个测试用例（ ）。

A.1

B.2

C.3

D.4

覆盖方法区别

- (1) 语句覆盖只关心代码行的执行。因此，语句覆盖是很弱的逻辑覆盖。
- (2) 判定覆盖只关心整个判定表达式的值。判定覆盖比语句覆盖强，但是对程序逻辑的覆盖程度仍然不高。
- (3) 条件覆盖通常比判定覆盖强，因为它使判定表达式中每个条件都取到了两个不同的结果，判定覆盖却只关心整个判定表达式的值。
- (4) 有时判定/条件覆盖也并不比条件覆盖更强。

覆盖方法区别

(5) 满足条件组合覆盖标准的测试数据，也一定满足判定覆盖、条件覆盖和判定/条件覆盖标准。因此，条件组合覆盖是前述几种覆盖标准中最强的。

(6) 但是，满足条件组合覆盖标准的测试数据并不一定能使程序中的每条路径都执行到。所以我们还需要用其他的测试方法(如黑盒法)作补充。

路径覆盖

路径覆盖就是设计所有的测试用例，来覆盖程序中的所有可能的执行路径。

基本路径测试法是在程序控制流图的基础上，通过分析控制构造的环路复杂性，导出基本可执行路径集合，从而设计测试用例的方法。设计出的测试用例要保证被测试程序的每个可执行语句至少被执行一次。

基本路径测试法

基本路径测试法：在程序控制流图的基础上，通过分析控制构造的环路复杂性，导出基本可执行的路径集合，从而设计测试用例的方法。

在基本路径测试中，设计出的测试用例要保证在测试中程序的每条可执行语句至少执行一次。在基本路径法中，需要使用程序的控制流图进行可视化表达。

程序的控制流图是描述程序控制流的一种图示方法。控制流图是退化的程序流程图，图中每个处理都退化成一个节点，流线变成连接不同节点的有向弧。

圆圈称为控制流图的一个结点，表示一个或多个无分支的语句或源程序语句；

箭头称为边或连接，代表控制流。

基本路径测试法-控制流程图

在选择或多分支结构中，分支的汇聚处应有一个**汇聚节点**；

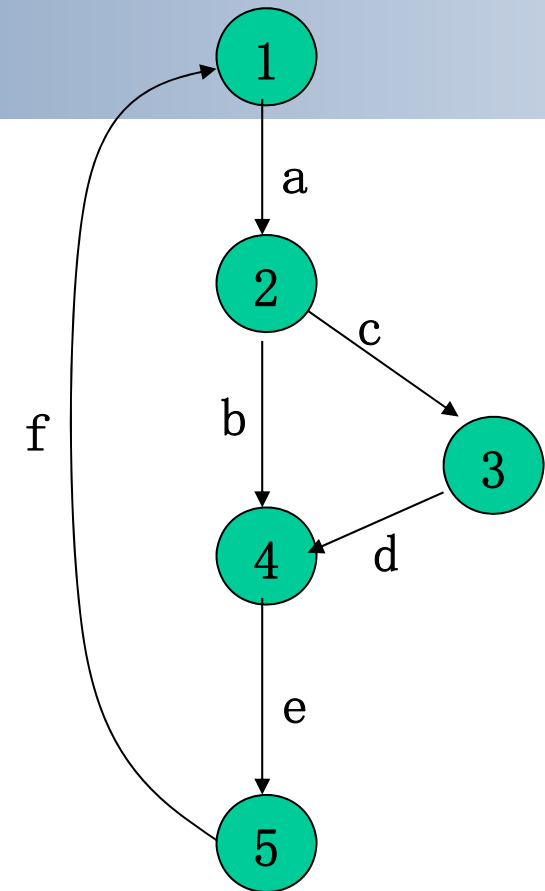
边和结点圈定的区域叫做**区域**，当对区域计数时，**图形外**的区域也应记为一个**区域**。

控制流程图：一种表示程序控制结构的图形工具，基本元素是**节点**、**判定**和**过程块**。

过程块：不能由判定、节点分开的一组语句。

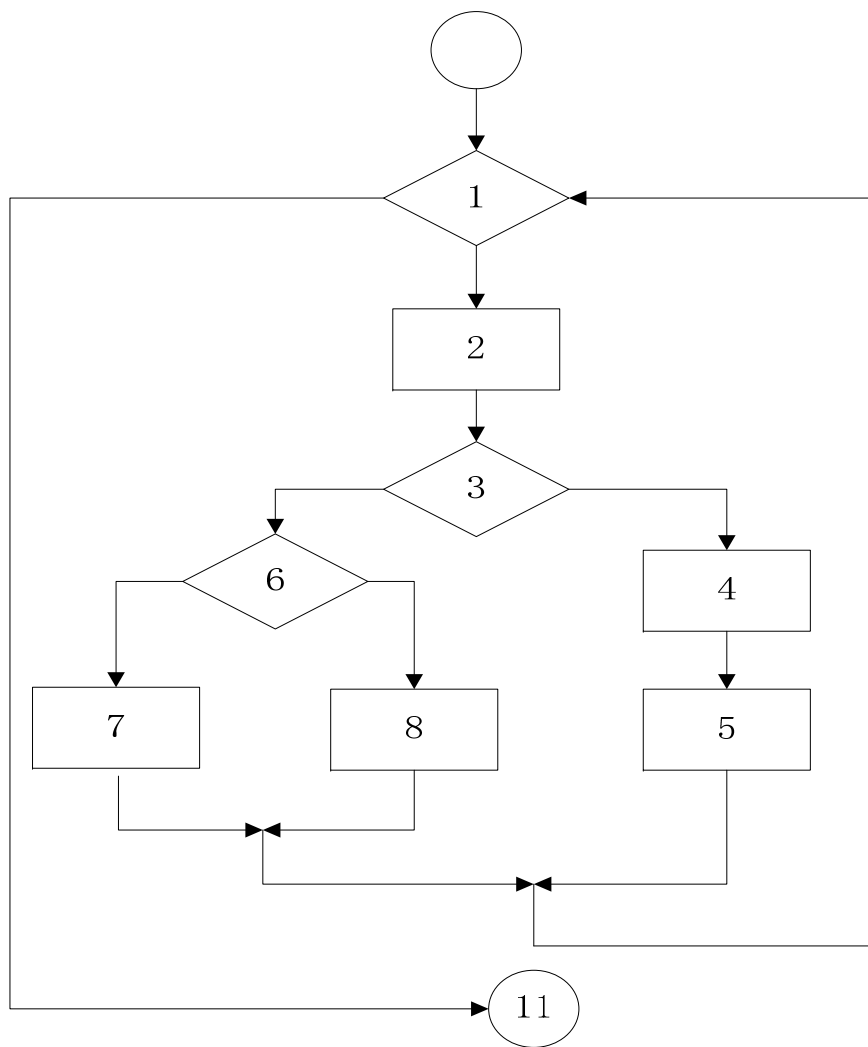
判定：判定条件处，一般此处有分支。

节点：控制流可以汇合。

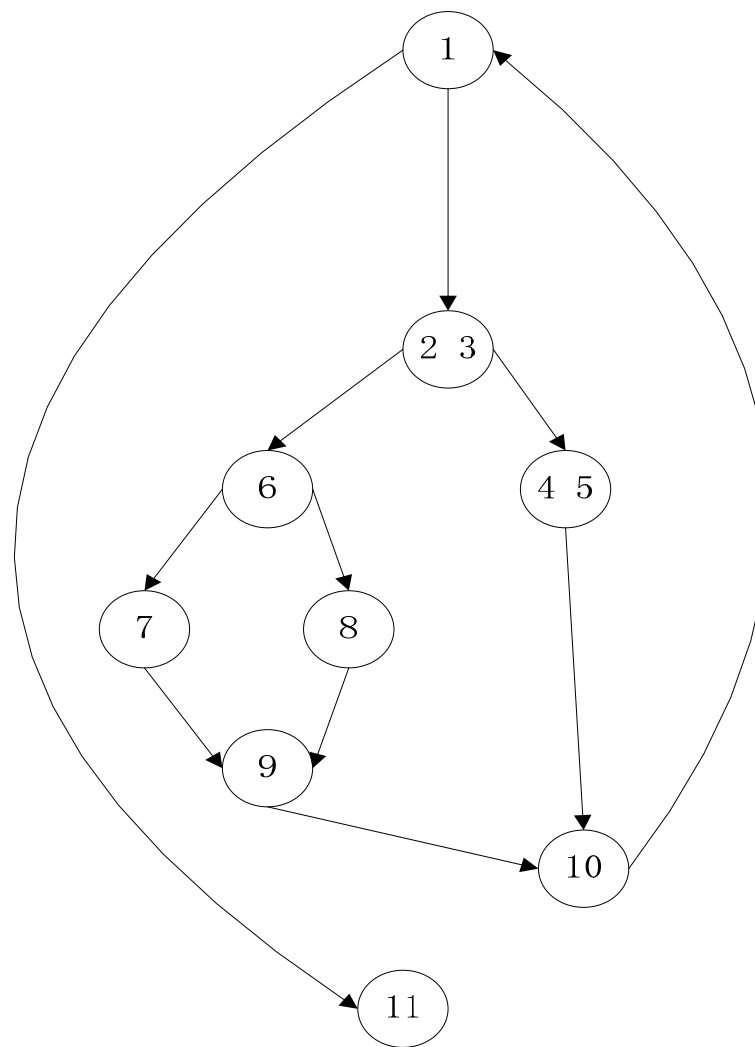


1、3、5 -过程块
2-判定
4-节点

基本路径测试法



(a) 程序流程图



(b) 控制流程图

基本路径测试是在**程序控制流图**的基础上，通过分析控制构造的环路复杂性，导出**基本可执行路径集合**，从而**设计测试用例**的方法。

基本路径测试主要包含以下四个方面：

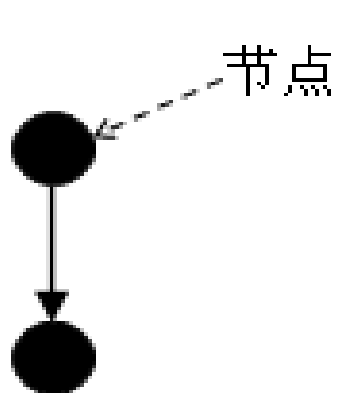
(1) **绘制出程序的控制流程图**，根据程序流程图，绘制出程序的控制流图。

(2) **计算程序环路复杂度**。环路复杂度是一种为程序逻辑复杂性提供定量测度的软件度量，将该度量用于计算程序的基本独立路径数目，这是程序中每个可执行语句至少执行一次所必须的最少测试用例数。

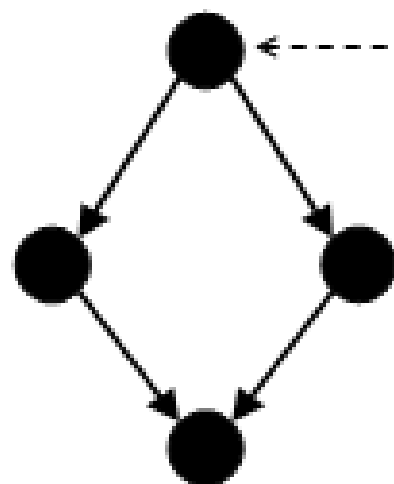
(3) **找出独立路径**。通过程序的控制流图导出基本路径集，列出程序的独立路径。

(4) **设计测试用例**。根据程序结构和程序环路复杂性设计用例输入数据和预期结果，确保基本路径集中的每一条路径的执行。

流程基本图元

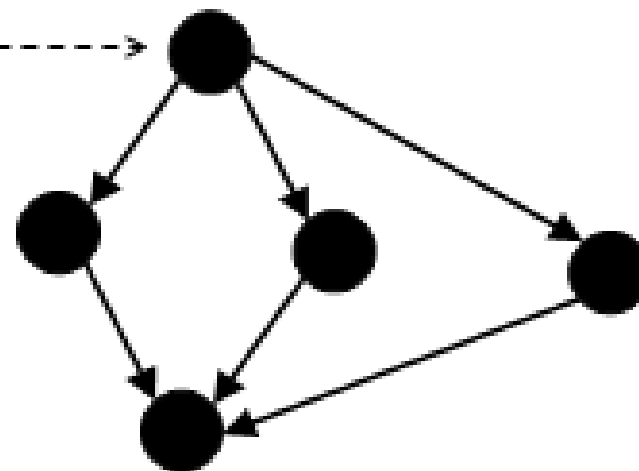


顺序

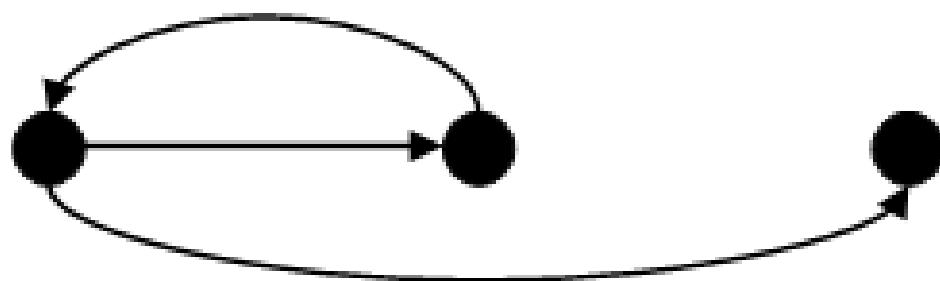


If-then-else

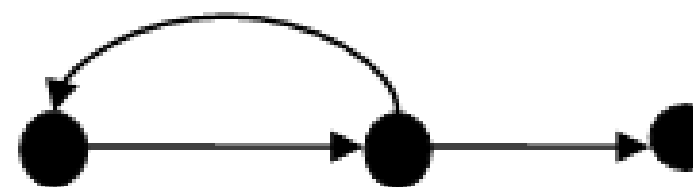
判断节点



Case



Do-while



Do-until

基本路径测试法

基本路径测试法适用于模块的详细设计及源程序。

其步骤如下：

- (1) 以详细设计或源代码为基础，导出程序的**控制流图**；
- (2) 计算得出**控制流图G**的环路复杂度 **$V(G)$** ；
- (3) 确定**线性无关的路径的基本集**；
- (4) 生成**测试用例**，确保基本路径集中每条路径的执行。

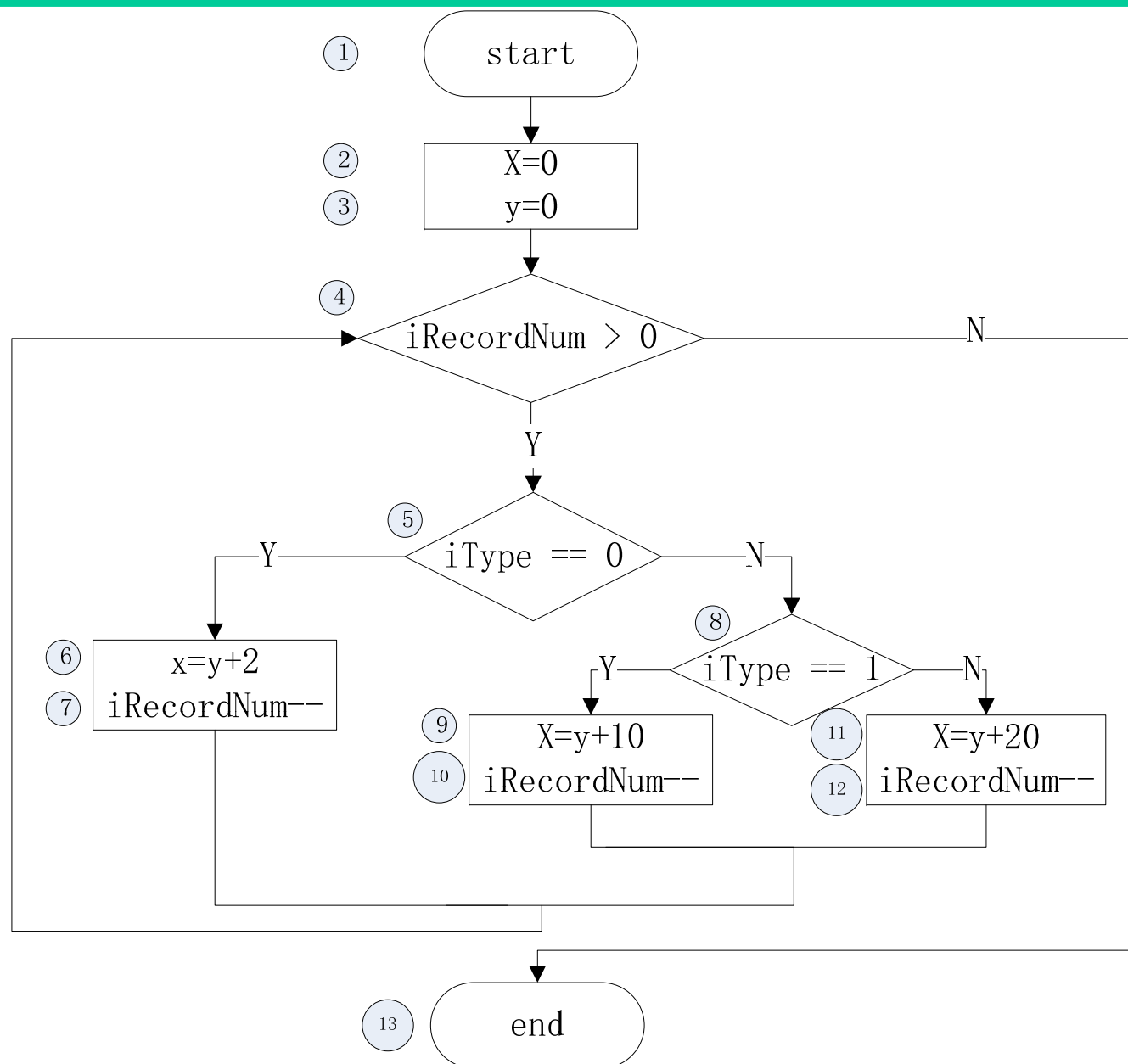
基本路径测试法

程序的环路复杂性即**McCabe**复杂性度量，从程序的环路复杂性可导出程序基本路径集合中的独立路径条数，确保程序中每个可执行语句至少执行一次所必须的测试用例数目的上界。

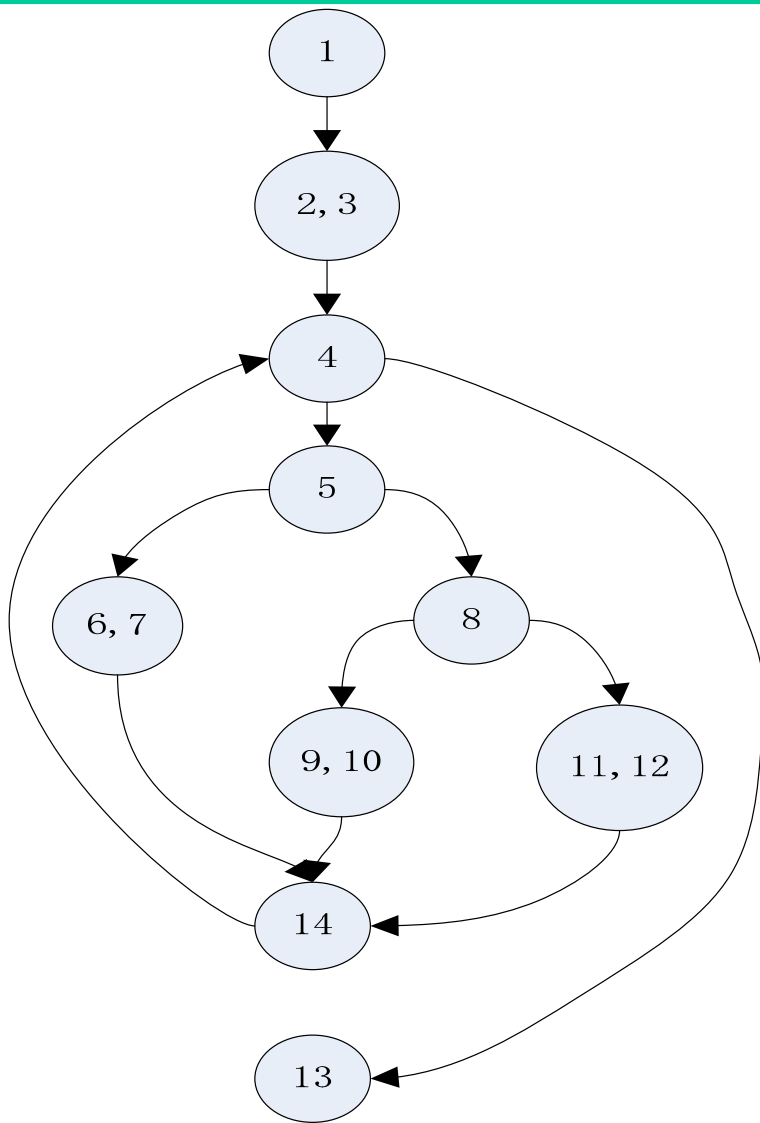
独立路径：指包括一组以前没有处理过的语句或条件的一条路径。

控制流图：一条独立路径是至少包含有一条在其他独立路径中从未有过的边的路径。

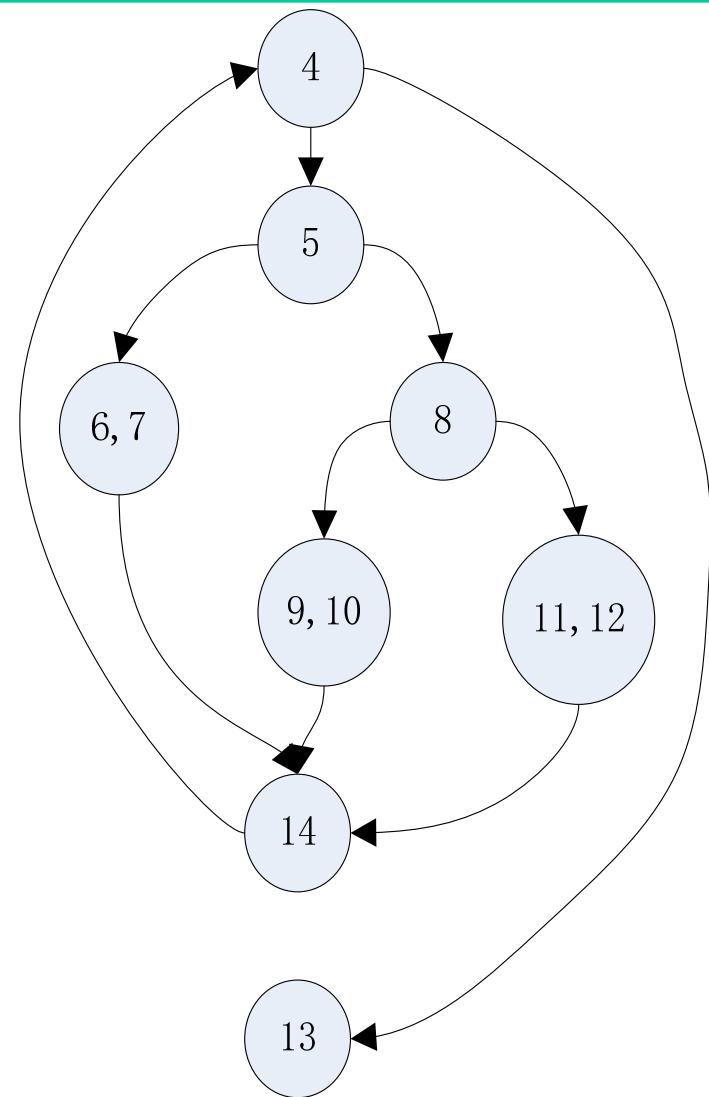
基本路径测试法



基本路径测试法



(a)



(b)

基本路径测试法

(2) 计算控制流图 **G** 的环路复杂性

$V(G)$ = 区域数目，区域由边界和节点围起来的形状构成，外部区域也作一个区域。

$$V(G)=4$$

或： **$V(G)=3$** （判断节点数） **$+1=4$** ；

或： **$V(G)=12$** （边界个数） **-10** （节点个数） **$+2=4$** 。

(3) 确定独立路径（用代码行号表示）

根据控制流图计算得到的环路复杂性，可知该程序的基本路径集中的路径条数为 **4**，

基本路径测试法

具体路径描述如下。

路径 1: 4→13

路径 2: 4→5→6, 7→14→4→13

路径 3: 4→5→8→9, 10→14→4→13

路径 4: 4→5→8→11, 12→14→4→13

(4) **设计测试用例**，确保基本路径集中的每条路径都被执行

用例1: iRecordNum=0 iType=0 路径1 x=0 y=0

用例2: iRecordNum=1 iType=0 路径2 x=2 y=0 iRecordNum=0

用例3: iRecordNum=1 iType=1 路径3 x=10 y=0 iRecordNum=0

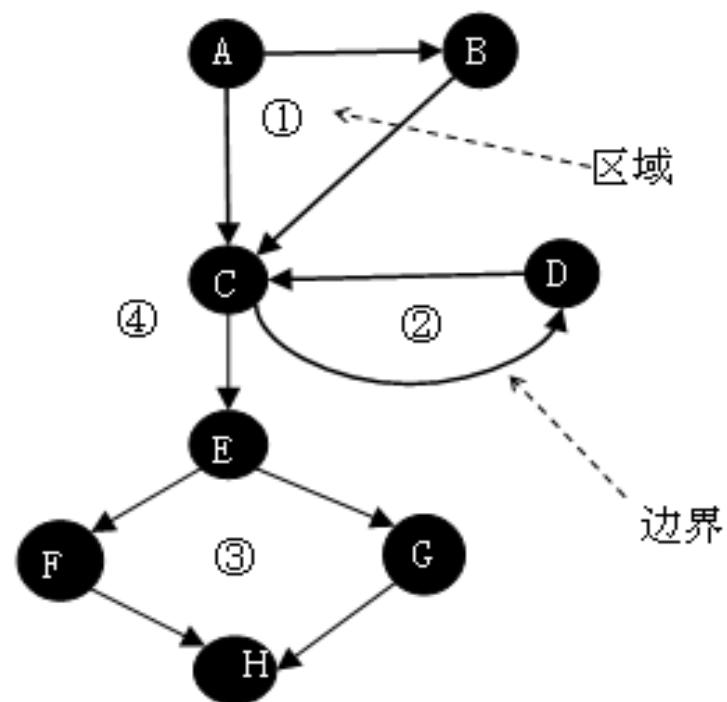
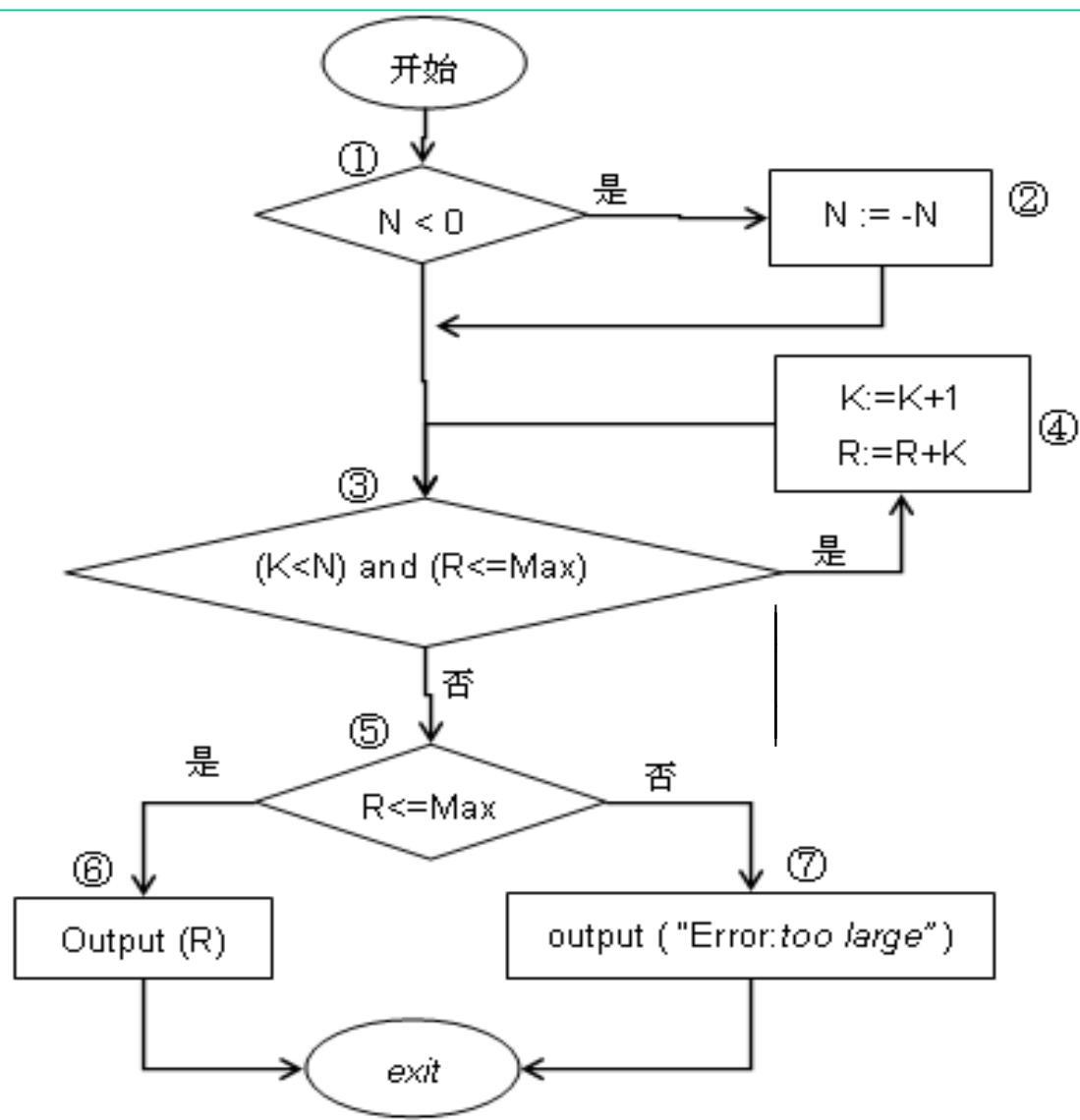
用例4: iRecordNum=1 iType=2 路径3 x=20 y=0 iRecordNum=0

基本路径测试法

```
1 int sort(int iRecordNum, int iType) {  
2     int x = 0;  
3     int y = 0;  
4     while (iRecordNum > 0) {  
5         if(iType == 0) {  
6             x=y+2;  
7             iRecordNum--;  
8         } else if(iType == 1)  
9             x =y+10;  
10        else  
11            x=y+20;  
12    }  
13    return x;
```

此代码有错，试试用测试
用例来找出程序中的错误

基本路径测试法

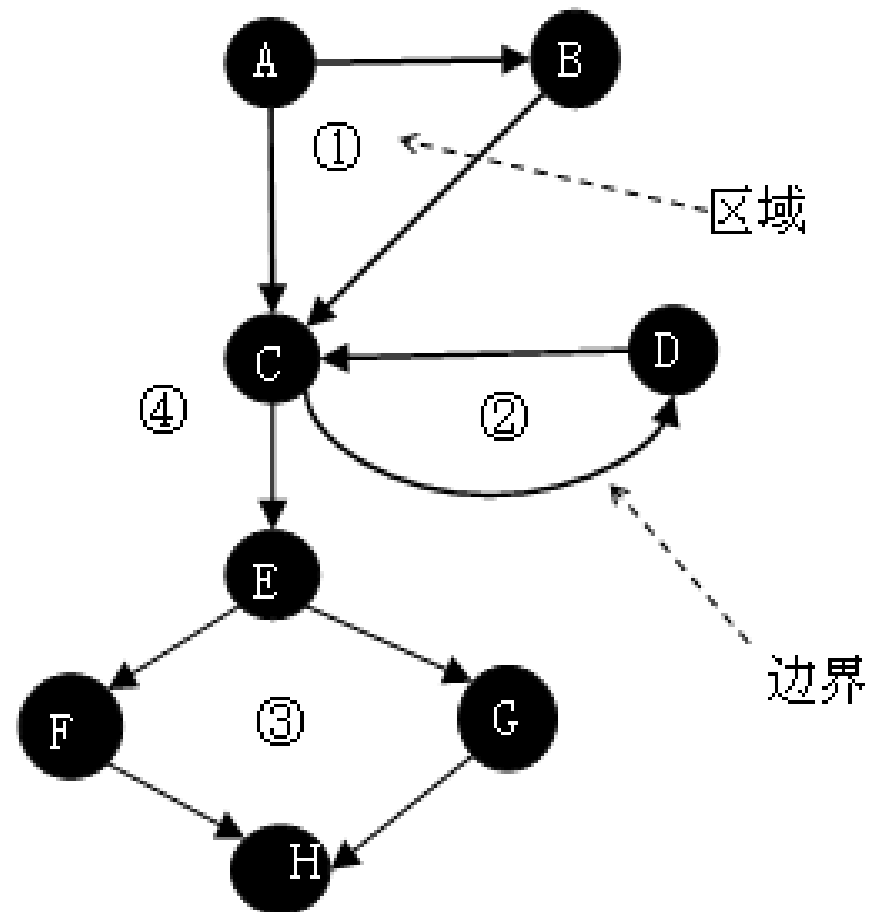


$V(G) = \text{区域数目}$

$V(G) = \text{边界数目} - \text{节点数目} + 2$

$V(G) = \text{判断节点数目} + 1$

示例计算结果: $V(G) = 4$



基本路径测试法

A—C—E—F—H

➡ $N=0$, $Max = 10$

A—C—E—G—H

➡ $N=0$, $Max = -1$

A—B—C—D—C—E—F—H

➡ $N=-2$, $Max = 10$

A—B—C—D—C—E—G—H

➡ $N=-10$, $Max = 10$

```

4 public int average(int value[], int minimum, int maximum)
5 {
6     int i = 0;
7     int inputNumber = 0;
8     int validNumber = 0;
9     int sum = 0;
10
11     while ((value[i] != -999) && (inputNumber < 100))
12     {
13         inputNumber++;
14         if ((value[i] >= minimum) && (value[i] <= maximum))
15         {
16             validNumber++;
17             sum = sum + value[i];
18         }
19         else break;
20         i++;
21     }
22
23     if validNumber > 0
24         average = sum/validNumber;
25     else
26         average = -999;
27 }

```

环复杂度：

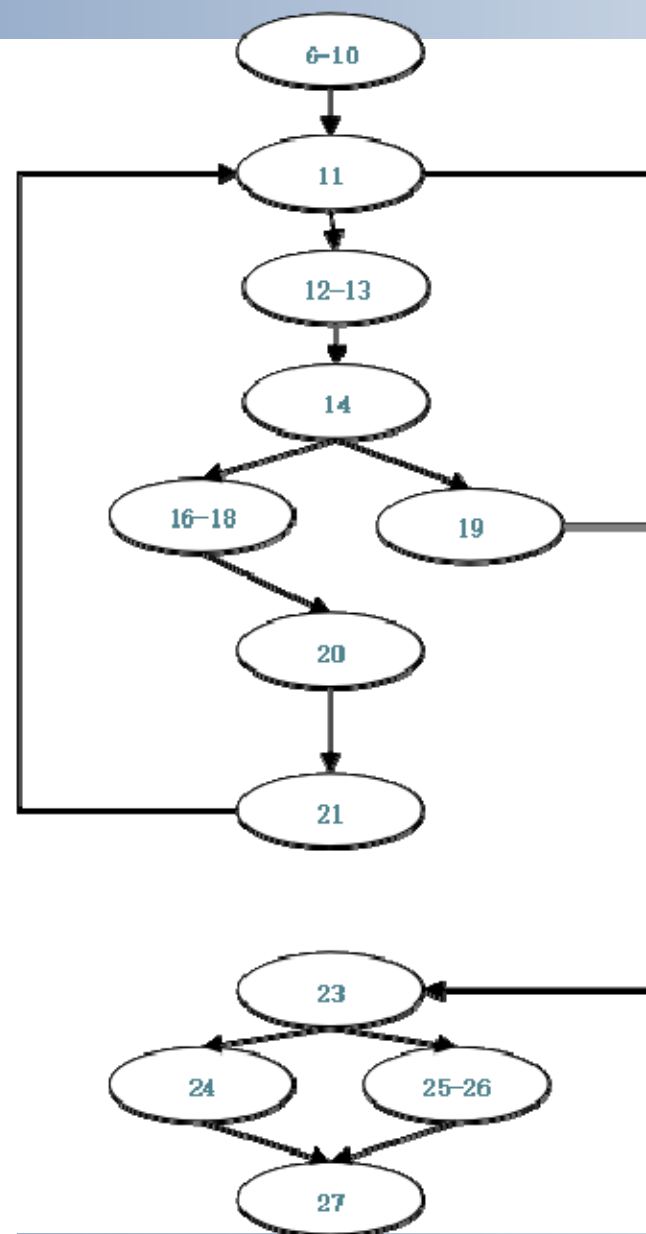
二值判定节点个数 + 1 = 3+1=4

边的数目-节点的数目 + 2 = 14-12+2=4

1、6-10→11→12-13→14→16-18→20→21→11→23→24→27 相应的测试用例：测试输入 = ((10, -999), 0, 100) 预期结果= 10

2、6-10→11→12-13→14→19→23→24→27 相应的测试用例：测试输入 = ((-10, -999), 0, 100) 预期结果= -999

3、6-10→11→23→25-26→27 相应的测试用例：测试输入 = ((-999), 0, 100) 预期结果= -999



代码级测试模式

- （1）在测试中，可采取先静态再动态的组合方式，先进行代码检查和静态结构分析，再进行覆盖测试；
- （2）利用静态分析的结果作为引导，通过代码检查和动态测试的方式对静态分析的结果做进一步确认；
- （3）覆盖测试是白盒测试的重点，一般可使用基本路径测试法达到语句覆盖标准，对于软件的重点模块，应使用多种覆盖标准衡量测试的覆盖率；
- （4）在不同的测试阶段测试重点不同，在单元测试阶段，以代码检查、覆盖测试为主，在集成测试阶段，需要增加静态结构分析等，在系统测试阶段，应根据黑盒测试的结果，采用相应的白盒测试方法。

黑盒白盒区别

白盒测试和黑盒测试是两类软件测试方法，传统的软件测试活动基本上都可以划分到这两类测试方法中。

白盒测试	黑盒测试
考察程序逻辑结构	不涉及程序结构
用程序结构信息生成测试用例	用软件规格说明书生成测试用例
主要适用于单元测试和集成测试	可适用于从单元测试到系统验收测试
对所有逻辑路径进行测试	某些代码段得不到测试

2、白盒法又称为逻辑覆盖法，主要用于（ ）。

A. 确认测试

B. 系统测试

C. α 测试

D. 单元测试

单元测试（Unit Testing） 又称模块测试，是对软件设计的最小单元进行功能、性能、接口和设计约束等正确性进行检验。

单元测试主要采用白盒测试技术。

白盒测试和黑盒测试各有侧重点，不能相互取代，在实际测试活动中，这两种测试方法不是截然分开的。

通常在白盒测试中交叉着黑盒测试，黑盒测试中交叉着白盒测试。相对来说，白盒测试比黑盒测试成本要高得多，它需要测试在可以被计划前产生源代码，并且在确定合适数据和决定软件是否正确方面需要花费更多的工作量。

在实际测试活动中，应当尽可能使用可获得的软件规格从黑盒测试方法开始测试计划，白盒测试计划应当在黑盒测试计划已成功通过之后再开始，使用已经产生的流程图和路径判定。路径应当根据黑盒测试计划进行检查并且决定和使用额外需要的测试。

灰盒测试是介于白盒测试与黑盒测试之间的测试。灰盒测试关注输出对于输入的正确性，同时也关注内部表现，但这种关注不像白盒测试那样详细、完整。灰盒测试结合了白盒测试和黑盒测试的优点，相对于黑盒测试和白盒测试而言，投入的时间相对较少，维护量也较小。灰盒测试考虑了用户端、特定的系统和操作环境，主要用于多模块的较复杂系统的集成测试阶段。灰盒测试既使用被测对象的整体特性，又使用被测对象的内部具体实现，即它无法知道函数内部具体内容，但是可以知道函数之间的调用。灰盒测试重点检验软件系统内部模块的接口。

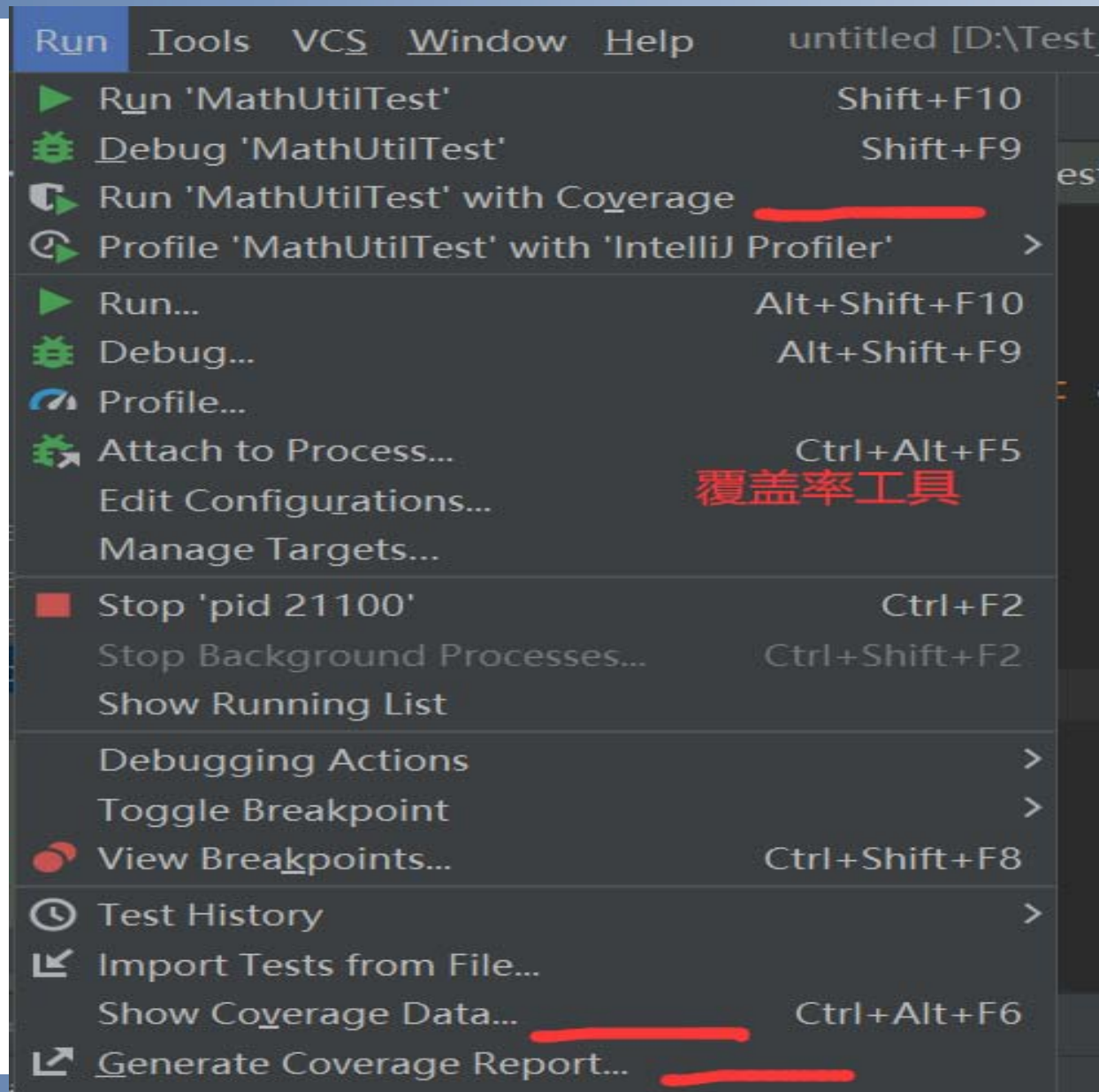
灰盒测试能够有效地发现黑盒测试的盲点，可以避免过渡测试，能够及时发现没有来源的更改，行业门槛比白盒测试低。但是灰盒测试不适用简单的系统，相对于黑盒测试来说门槛较高，测试没有白盒测试那么深入。

代码覆盖率分析工具

- ❑ **JaCoCo**通过对编译后的**Java class**文件进行统计代码的插装，测试覆盖率的收集与报告。
- ❑ **BullseyeCoverage**是一种获取**C/C++**的代码覆盖率的工具，支持分支覆盖率（**Branch Coverage**）的分析
- ❑ **Testwell CTC++**, **Parasoft Jtest/C++Test**, **CoverageMeter**, **Jcov**, **CodeCover**, **Coverage.py**.....

JUNIT——分析单元测试覆盖率

idea



JUNIT——分析单元测试覆盖率

Current scope: all classes | com.dw.service

Coverage Summary for Class: MathUtil (com.dw.service)

Class	Class, %	Method, %	Branch, %	Line, %
MathUtil	100% (1/1)	100% (4/4)	50% (9/18)	64.3% (18/28)

```
7  
8 public int jTest(int iRecordNum, int iType) {  
9
```

```
0     int x = 0;  
1     int y = 0;  
2     while (iRecordNum > 0) {  
3         if (iType == 0) {  
4             x = y + 2;  
5  
6         } else if (iType == 1)  
7             x = y + 10;  
8         else  
9             x=y + 20;  
0  
1         iRecordNum--;  
2     }  
3     return x;  
4 }  
5
```

```
1 package com.dw.service;  
2 public class MathUtil {  
3     public static int max(int a,  
4         if(a > b){  
5             if(a > c){  
6                 return a;  
7             }else{  
8                 return c;  
9             }  
10        }else{  
11            if(b > c){  
12                return b;  
13            }else{  
14                return c;  
15            }  
16        }  
17    }
```